JURONG PIONEER JUNIOR COLLEGE

JC2 Theory Practice Set 1 (2026)

COMPUTING 9569/01 Paper 1 (Written)

SUGGESTED MARK SCHEME

General marking notes

- One mark is awarded for each marking point unless stated otherwise. Do not award more than the maximum for any part.
- Accept any correct and unambiguous pseudo-code, including Python-style code, unless the question asks for a specific form (for example, a recursive solution). Minor syntax slips should not be penalised if the logic is clear.
- Credit valid alternative answers that are not listed if they are correct, relevant to the context, and at the same depth as the listed points.
- Explanations must be applied to the scenario where the question asks for it ("refer to the data", "with reference to the app"). A generic answer earns the identification mark only.
- Table descriptions: solid underline = primary key, dashed underline = foreign key. In this scheme, a double underline marks an attribute that is both part of the primary key and a foreign key.

Paper blueprint

Q	Context	Main syllabus learning outcomes	Marks
1	Bicycle-sharing unlock rules	1.1.3, 1.1.4 decision tables; pseudo-code; 2.4.4 test data	13
2	Number base conversion	2.2.5 to 2.2.7 recursion and the call stack; 2.4.3 run-time errors; 3.1, 3.2 bases and Unicode	14
3	Music playlist	1.3.2 dynamic memory allocation; 1.3.5 linked list insert and delete; binary search on a list	12
4	Library book codes	1.2.3 hash table search with linear probing; 3.2.2 ASCII in a hash function	10
5	Analytics records	1.2.1 to 1.2.5 bubble and merge sort, binary search, Big-O	12
6	FitTrack fitness app	2.5.1 to 2.5.5 classes, encapsulation, inheritance, polymorphism, class diagram	11
7	PawsCare vet clinic	3.3.2 to 3.3.8 keys, 2NF and 3NF, ER diagram, SQL aggregation, NoSQL; 3.3.11 backup and archive	15
8	QueueNow polyclinic queue	4.1.4 to 4.1.6 packet switching, sockets; 4.3.3 digital signatures; 3.4.3 ethics	13
		Total	**100**

Revised after the 2026 A-level Paper 2 (Oct/Nov 2026), which tested a linear queue and stack, a class with getters, file handling with validation, SQL table creation and a join query, and an array-based binary search tree (insertion, in-order and a recursive check). The binary search tree and free space list questions were replaced with a dynamic linked list and a hash table, and the SQL part was reduced in favour of backup and archive. Topics reinforced from the 2026 JPJC Prelim P1 review: SQL with GROUP BY, HAVING and ORDER BY; linked list pointer changes; test data; private/public notation and overridden methods in class diagrams; instantiation; digital signature verification steps.

Question 1: Decision table (CycleGo)

(a) — 4 marks

account has an unpaid penalty	Y	Y	Y	Y	N	N	N	N
identity verification completed	Y	Y	N	N	Y	Y	N	N
wallet balance is at least $5.00	Y	N	Y	N	Y	N	Y	N
Account suspended	X	X	X	X				
Verify identity							X	X
Top up required						X		
Bicycle unlocked					X			

1 mark: rules 1 to 4 (penalty = Y) all give Account suspended only

1 mark: rule 5 (N, Y, Y) gives Bicycle unlocked only

1 mark: rule 6 (N, Y, N) gives Top up required only

1 mark: rules 7 and 8 (N, N, Y/N) give Verify identity only

Each rule must have exactly one action ticked. A rule with two actions loses the mark for that rule.

(b) — 2 marks

account has an unpaid penalty	Y	N	N	N
identity verification completed	-	Y	Y	N
wallet balance is at least $5.00	-	Y	N	-
Account suspended	X			
Verify identity				X
Top up required			X	
Bicycle unlocked		X		

1 mark: rules 1 to 4 merged into one rule (Y, -, -) with Account suspended

1 mark: rules 7 and 8 merged into one rule (N, N, -) with Verify identity, and the two remaining rules (N, Y, Y) and (N, Y, N) kept with correct actions

Column order may differ. Accept a blank or "-" for "don't care".

(c) — 4 marks

```
INPUT HasPenalty
INPUT IsVerified
INPUT Balance
IF HasPenalty = TRUE THEN
   OUTPUT "Account suspended"
ELSE
   IF IsVerified = FALSE THEN
      OUTPUT "Verify identity"
   ELSE
      IF Balance >= 5.00 THEN
         OUTPUT "Bicycle unlocked"
      ELSE
         OUTPUT "Top up required"
      ENDIF
   ENDIF
ENDIF
```

1 mark: input of all three identifiers

1 mark: penalty tested first (or in a way that cannot be overridden) and outputs Account suspended

1 mark: when there is no penalty, not verified outputs Verify identity

1 mark: verified with Balance >= 5.00 outputs Bicycle unlocked, otherwise Top up required (correct boundary: >= not >)

Accept ELSEIF/CASE, or separate IF statements with compound conditions if exactly one message is output for every combination.

(d) — 3 marks

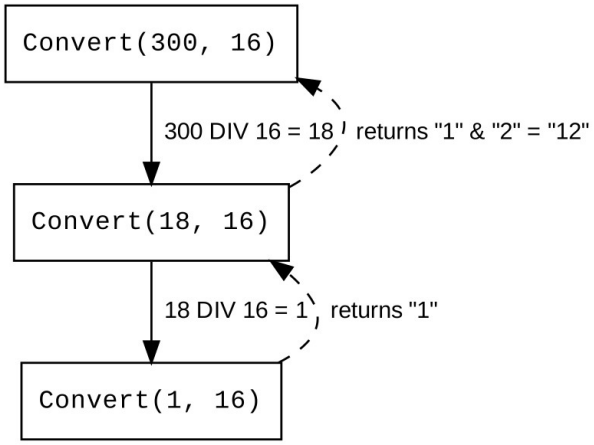
Type	Example value	Reason
Normal	50 (any whole number from 11 to	within the valid range, should be

	99)	accepted
Extreme	10 or 100	on the boundary of the valid range, should be accepted
Abnormal	e.g. 5, 101, -20, 25.50, "ten"	outside the range / not a whole number / wrong data type, should be rejected

1 mark per row: value and reason must both be correct

Do not accept 10 or 100 as normal. Accept 9 or 101 as abnormal (some texts call these "boundary invalid").

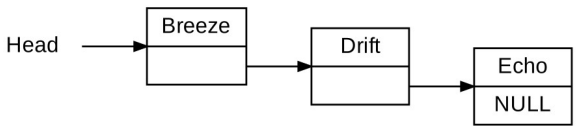
Question 2: Recursion and number bases

Part	Suggested answer and marking points	Marks
(a)	 Convert(300, 16) calls Convert(18, 16), which calls Convert(1, 16). Convert(1, 16) returns "1" (base case, 1 < 16). Convert(18, 16) returns "1" & Symbols[2] = "12". Convert(300, 16) returns "12" & Symbols[12] = "12C". 1 mark: correct sequence of three calls with parameters (300, 16), (18, 16), (1, 16) 1 mark: Convert(1, 16) returns "1" and Convert(18, 16) returns "12" 1 mark: final value "12C"	**3**
(b)(i)	Converts a positive (denary) integer N into a string representing the number in base B (for example binary, octal or hexadecimal). *Must mention the conversion to base B. "Converts to hexadecimal" only: 0.*	**1**
(b)(ii)	"65B0" (6 × 4096 + 5 × 256 + 11 × 16 + 0 = 26032)	**1**
(b)(iii)	ASCII uses 7 bits, giving only 128 codes (0 to 127), which cover English letters, digits and symbols only. 26032 is outside this range, so there is no ASCII code for the character (Unicode supports characters from all writing systems).	**1**
(c)	Each call (300, 16), (18, 16), (1, 16) pushes a stack frame containing its parameters N and B and the return address onto the call stack. [1] When the base case is reached, the frames are popped in reverse order (LIFO). Each returned value is passed back to the calling frame, which concatenates its own digit before returning. [1]	**2**
(d)	With B = 1, N DIV 1 = N, so N never decreases and the base case N < 1 is never reached. The function calls itself without end (infinite recursion), each call adding a frame until the stack runs out of memory (stack overflow) and the program crashes. [1] Run-time error. [1] *Do not accept syntax error or logic error. Accept "RecursionError" if described as occurring at run time.*	**2**

Part	Suggested answer and marking points	Marks

(e)	`FUNCTION ConvertIter(N : INTEGER, B : INTEGER) RETURNS STRING`
	`  DECLARE Symbols, Result : STRING`
	`  Symbols ← "0123456789ABCDEF"`
	`  Result ← ""`
	`  REPEAT`
	`  Result ← Symbols[N MOD B] & Result`
	`  N ← N DIV B`
	`  UNTIL N = 0`
	`  RETURN Result`
	`ENDFUNCTION`
	1 mark: Result initialised to an empty string (and Symbols available)
	1 mark: loop that repeats until N = 0 (or WHILE N > 0)
	1 mark: Symbols[N MOD B] placed in front of Result (or appended and Result reversed at the end)
	1 mark: N ← N DIV B inside the loop and Result returned after the loop
	A recursive answer scores 0. Appending without reversing loses the third mark.
		4

Question 3: Linked list with dynamic nodes (playlist)

Part	Suggested answer and marking points	Marks
(a)	Advantage: memory is allocated only when a song is added and released when it is removed, so the playlist can grow to any length (no fixed maximum) and no memory is wasted on unused array elements. [1] Disadvantage: each node needs extra memory for the Next pointer; or a song cannot be accessed directly by its position, so the list must be traversed from Head; or memory may run out at run time. [1] *One advantage and one disadvantage, each applied to the playlist.*	**2**
(b)	Insert "Drift": a new node is created for Drift; Drift.Next is set to point to Echo (Breeze's old Next), then Breeze.Next is set to point to Drift. [1] Delete "Aurora": Head is set to point to Breeze (Aurora.Next) and the Aurora node is released. [1] Delete "Halo": Echo.Next is set to NULL (Halo.Next) and the Halo node is released. [1]  *Award the mark for an operation only if the pointer changes are correct and in a workable order. A final list Head → Breeze → Drift → Echo → NULL with no description of the pointer changes: 1 mark only.*	**3**

(c)	`PROCEDURE DeleteTitle(Target : STRING)`
	`  IF Head = NULL THEN`
	`  OUTPUT "Playlist is empty"`
	`  ELSE`
	`  IF Head.Title = Target THEN`
	`  Head ← Head.Next  // delete first node`
	`  ELSE`
	`  Prev ← Head`
	`  Curr ← Head.Next`
	`  WHILE Curr <> NULL AND Curr.Title <> Target`
	`  Prev ← Curr`
	`  Curr ← Curr.Next`
	`  ENDWHILE`
	`  IF Curr = NULL THEN`
		5

<table>
<tr><th>Part</th><th>Suggested answer and marking points</th><th>Marks</th></tr>
<tr><td></td><td>

```
            OUTPUT "Song not found"
         ELSE
            Prev.Next ← Curr.Next     // bypass the node
         ENDIF
      ENDIF
   ENDIF
ENDPROCEDURE
```

1 mark: checks for an empty playlist (Head = NULL) and outputs a message
1 mark: deletes the first node correctly by updating Head when Head.Title = Target
1 mark: traverses with a previous and a current pointer until Target or NULL is reached
1 mark: outputs a message when the end of the list is reached without finding Target
1 mark: relinks Prev.Next ← Curr.Next
Accept a single pointer that looks ahead (Curr.Next.Title = Target). Releasing the deleted node explicitly is not required.
</td><td></td></tr>
<tr><td>(d)</td><td>Binary search needs direct access to the middle element by its index. A linked list has no index: the only way to reach the middle node is to follow the pointers from Head. [1]
Finding each middle node takes O(n) steps, so the search is no faster than a linear search (O(n)) and loses the O(log n) benefit. [1]</td><td>2</td></tr>
</table>

Question 4: Hash table with linear probing (library)

<table>
<tr><th>Part</th><th>Suggested answer and marking points</th><th>Marks</th></tr>
<tr><td>(a)</td><td>ASC("A") + ASC("B") + ASC("1") + ASC("2") = 65 + 66 + 49 + 50 = 230 [1]
230 MOD 11 = 230 − 220 = 10 [1]</td><td>2</td></tr>
<tr><td>(b)</td><td>

Index	0	1	2	3	4	5	6	7	8	9	10
Code	BA21	PD44	KQ30		MX07			CS21	ZE15	HT09	AB12

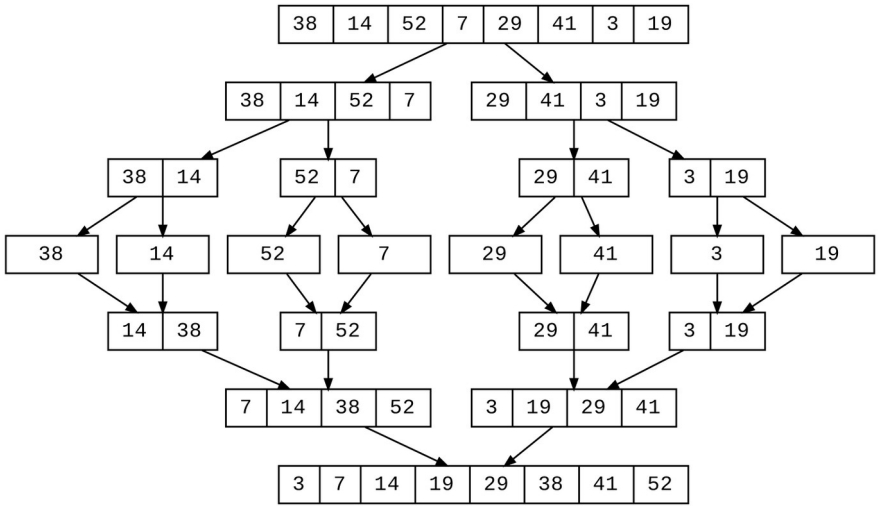
1 mark: AB12, CS21, MX07, KQ30 and ZE15 at their home addresses 10, 7, 4, 2 and 8
1 mark: BA21 at 0 and PD44 at 1 (probing from 10 wraps around to 0)
1 mark: HT09 at 9; indices 3, 5 and 6 empty
</td><td>3</td></tr>
<tr><td>(c)</td><td>

```
FUNCTION FindCode(Code : STRING) RETURNS INTEGER
   DECLARE Index, Count : INTEGER
   Index ← Hash(Code)
   Count ← 0
   WHILE HashTable[Index] <> "" AND Count < 11
      IF HashTable[Index] = Code THEN
         RETURN Index
      ENDIF
      Index ← (Index + 1) MOD 11
      Count ← Count + 1
   ENDWHILE
   RETURN -1
ENDFUNCTION
```

1 mark: starts at Index ← Hash(Code)
1 mark: returns Index when HashTable[Index] = Code
1 mark: moves to (Index + 1) MOD 11 to probe the next element, wrapping from 10 to 0
1 mark: stops and returns -1 when an empty element is reached or all 11
</td><td>4</td></tr>
</table>

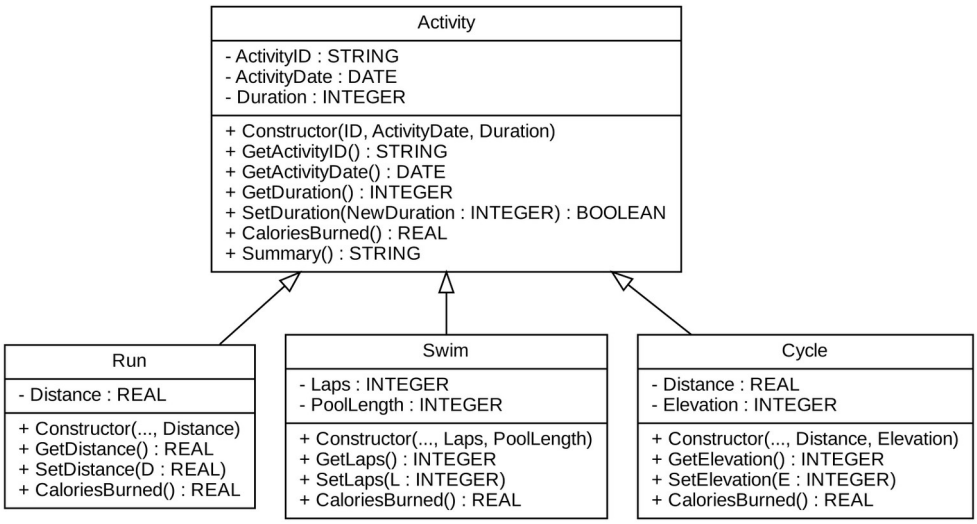
	elements have been checked *A linear search of the whole array from index 0 scores 0. A loop that never stops when the table is full loses the last mark.*	
(d)	The hash value depends only on the sum of the ASCII codes, so codes made of the same characters in a different order give the same hash value, for example AB12 and BA21 (both 230, hash 10). Codes with the same total, such as ZE15 and HT09 (261), also collide. *Accept any clear explanation that equal character sums give equal hash values, with an example from part (b).*	**1**

Question 5: Sorting and searching

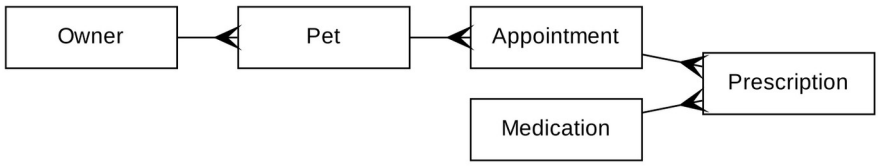
Part	Suggested answer and marking points	Marks
(a)	After pass 1: 14 38 7 29 41 3 19 52 [1] After pass 2: 14 7 29 38 3 19 41 52 [1] *If a candidate's bubble sort moves the smallest value to the front in each pass (working from the right), accept 3 38 14 52 7 29 41 19 then 3 7 38 14 52 19 29 41.*	**2**
(b)	 1 mark: the list is repeatedly split into halves until each sub-list has one element 1 mark: sub-lists are merged in pairs into sorted sub-lists: [14 38] [7 52] [29 41] [3 19], then [7 14 38 52] [3 19 29 41] 1 mark: final merge gives 3 7 14 19 29 38 41 52, with annotation that merging compares the front items of two sorted lists and takes the smaller each time	**3**
(c)	**Algorithm** / **Worst case** Bubble sort / $O(n^2)$ Insertion sort / $O(n^2)$ Quicksort / $O(n^2)$ Merge sort / $O(n \log n)$ 2 marks: all four correct. 1 mark: two or three correct.	**2**
(d)	Merge sort has a guaranteed worst-case time complexity of O(n log n), whatever the initial order of the records. [1] Quicksort can degrade to $O(n^2)$ (for example, when the data is already sorted or nearly sorted and a poor pivot is chosen). For 2 million records this is about 4×10^{12} comparisons compared with about 4×10^7, so the job may not finish within the time window. [1] *Also accept: merge sort is stable, so records with equal keys keep their order; or merge sort suits sorting data held in files that are too large for memory.*	**2**
(e)	Low = 0, High = 7, Mid = 3: 19 < 40, Low = 4 Mid = 5: 38 < 40, Low = 6	**2**

	Mid = 6: 41 > 40, High = 5. Low > High, so the search stops. 1 mark: values compared are 19, 38, 41 in that order 1 mark: 40 is not found (search ends when Low > High)	
(f)	10 (since $2^{10} = 1024 \geq 1000$)	**1**

Question 6: Object-oriented programming (FitTrack)

Part	Suggested answer and marking points	Marks
(a)	 Any six of the following, 1 mark each: • superclass Activity with ActivityID, ActivityDate and Duration, with data types • three subclasses Run, Swim and Cycle with the correct extra attributes (Run: Distance; Swim: Laps, PoolLength; Cycle: Distance, Elevation) • inheritance arrows from each subclass pointing to Activity (hollow triangle at Activity) • attributes shown as private (-) and methods as public (+) • constructor and getter methods in Activity, including a setter SetDuration • CaloriesBurned() and Summary() declared in Activity • CaloriesBurned() overridden in each of Run, Swim and Cycle *Attributes repeated in the subclasses that are already in Activity: do not award the second point. Arrows pointing from the superclass to the subclasses: do not award the third point.*	**6**
(b)	The same method call CaloriesBurned() is made on every object in Workouts, but each object runs its own overridden version: a Run object uses 62 × distance, a Swim object uses the laps and pool length, a Cycle object uses distance and elevation. The version used is decided at run time by the class of the object. [1] Benefit: the total can be calculated with one simple loop without checking the type of each object; a new type of activity (for example Hike) can be added with its own CaloriesBurned() without changing the calculation code. [1]	**2**
(c)	Duration is declared private, so code outside the class cannot assign a value to it directly. [1] Duration can only be changed through public methods (the constructor and SetDuration), which check that the value is an integer from 1 to 600 and reject the value (for example, return FALSE) if it is not. [1]	**2**
(d)	Instantiation is the creation of an object (an instance) from a class, for example creating a Run object for a 45-minute 8.5 km run: MyRun ← NEW Run("A001", 27/10/2026, 45, 8.5). *Must refer to creating an object from a class. "Declaring a class" or "an instance of a class" alone: 0.*	**1**

Question 7: Databases (PawsCare)

Part	Suggested answer and marking points	Marks
(a)	The primary key is the composite key (ApptID, MedCode), but some non-key attributes depend on only part of the key (partial dependency). [1] For example, ApptDate and PetID depend only on ApptID (A101, 2026-10-05, P01 is repeated in two rows), and MedName and UnitPrice depend only on MedCode (M03, Deworm tablet, 12.50 is repeated). [1]	**2**
(b)	Owner(OwnerID, OwnerName, OwnerPhone) Pet(PetID, PetName, Species, OwnerID) Appointment(ApptID, ApptDate, PetID) Medication(MedCode, MedName, UnitPrice) Prescription(ApptID, MedCode, Quantity) (ApptID and MedCode together form the primary key; each is also a foreign key) 1 mark: Owner and Medication correct 1 mark: Pet with OwnerID as foreign key 1 mark: Appointment with PetID as foreign key 1 mark: Prescription with composite primary key (ApptID, MedCode), both shown as foreign keys, and Quantity *Accept a surrogate key (for example PrescriptionID) for Prescription only if ApptID and MedCode are both shown as foreign keys.*	**4**
(c)	 1 mark: Owner to Pet and Pet to Appointment, both one-to-many 1 mark: Appointment to Prescription, one-to-many 1 mark: Medication to Prescription, one-to-many *A direct many-to-many relationship between Appointment and Medication scores 0 for the last two marks.*	**3**

(d)	`SELECT O.OwnerName, SUM(Pr.Quantity * M.UnitPrice) AS TotalCost`
	`FROM Owner AS O`
	`  INNER JOIN Pet AS P ON O.OwnerID = P.OwnerID`
	`  INNER JOIN Appointment AS A ON P.PetID = A.PetID`
	`  INNER JOIN Prescription AS Pr ON A.ApptID = Pr.ApptID`
	`  INNER JOIN Medication AS M ON Pr.MedCode = M.MedCode`
	`GROUP BY O.OwnerID, O.OwnerName`
	`HAVING SUM(Pr.Quantity * M.UnitPrice) > 50`
	`ORDER BY TotalCost DESC;`
	1 mark: SELECT OwnerName and SUM(Quantity * UnitPrice) AS TotalCost
	1 mark: all five tables joined correctly (JOIN ... ON, or join conditions in WHERE)
	1 mark: GROUP BY owner, HAVING (not WHERE) for the total > 50, and ORDER BY TotalCost DESC
	Checked in SQLite with the sample rows: returns Tan Wei Ling, 72.4. Table aliases are optional. Accept HAVING TotalCost > 50 (valid in SQLite).
		3

(e)	A backup is a copy of current data (for example, today's appointment and prescription records) kept separately so that the data can be restored if the original is lost, corrupted or attacked; it is made regularly and the original stays in use. [1] An archive is old data that is no longer in regular use (for example, appointment records from previous years) moved to separate long-term storage to free up the main system, but kept for reference or legal reasons. [1]	**2**

Part	Suggested answer and marking points	Marks
(f)	Any one, applied to the data: • flexible schema: documents in the same collection can have different fields, so records with different kinds of data can be stored without changing a fixed table structure • better suited to unstructured or semi-structured data such as images and free text • scales horizontally across many servers to handle large volumes such as X-ray images	**1**

Question 8: Networks, security and ethics (QueueNow)

Part	Suggested answer and marking points	Marks
(a)	The page is broken into packets. Each packet has a header containing the source and destination IP addresses and a sequence number. [1] Routers forward each packet independently towards the destination, possibly by different routes. At the phone, the packets are put back in order using the sequence numbers and missing or corrupted packets are requested again. [1]	**2**
(b)	Any three, in a sensible order, 1 mark each: • create a socket • bind the socket to the server's IP address and a port number, and listen for connection requests • accept a connection from a kiosk (waits until a kiosk connects), creating a connection socket • receive the registration request, process it and send the queue number back through the connection socket • close the connection and loop back to accept the next kiosk, serving one kiosk at a time (iterative)	**3**
(c)	Any three, 1 mark each: • the kiosk uses the queue server's public key (for example, from the server's digital certificate) to decrypt the digital signature, giving the message digest (hash) created by the server • the kiosk applies the same hash function to the message it received to produce a new digest • the kiosk compares the two digests • if they match, the message has not been altered, and it must have come from the queue server because only the server's private key could have produced a signature that decrypts correctly with its public key; if they do not match, the message is rejected *Using the kiosk's own private key, or the server's private key, to verify: do not award the first point.*	**3**
(d)	1 mark for each relevant issue explained in the polyclinic context, maximum 4. 1 further mark for a balanced point or a suitable safeguard (for example, a nurse reviews the AI's decision, or the AI is tested for bias before use). Possible issues: • accuracy and bias: if the training data under-represents some groups (elderly patients, children, people who type in another language), the AI may give them the wrong priority and delay urgent care • accountability: unclear who is responsible if a patient is harmed by a wrong priority (the polyclinic, the doctors or the AI vendor) • transparency: patients and staff may not understand why a priority was given, so decisions are hard to question or appeal • privacy: symptoms are sensitive personal data; under the PDPA the polyclinic needs consent, must use the data only for this purpose and must protect it	**5**

Part	Suggested answer and marking points	Marks
	from leaks • accessibility and the digital divide: patients who are less able to use a kiosk or describe their symptoms in writing may be disadvantaged • fairness and honesty: some patients may exaggerate symptoms to be seen sooner • impact on staff: triage nurses may be redeployed or lose skills, although they may also have more time for patient care • benefits to balance: faster and more consistent triage, shorter waiting times for urgent cases *Answers that list issues without explaining them in context: maximum 2.*	

END OF MARK SCHEME